



**UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS**

**FINAL PROJECT COMMERCIAL MANAGEMENT SYSTEM
QUERY IMPLEMENTATION ANALYSIS**

**Presented to:
ACMUD Web Development Java Spring Work Group**

**Present by:
Team 09**

**ACMUD Web Development Java Spring
Universidad Distrital Francisco José De Caldas
Ingeniería de Sistemas
November 26th 2025**

FINAL PROJECT COMMERCIAL MANAGEMENT SYSTEM QUERY IMPLEMENTATION ANALYSIS

1. Summary

System Overview The Commercial Management System is a backend application built using Java Spring Boot, designed to manage commercial operations including users, products, stores, and sales. The system utilizes a layered architecture comprising Controllers (REST API), Services (Business Logic), Repositories (Data Access), and Entities (Data Model).

ORM Architecture The project implements Object-Relational Mapping (ORM) using the Java Persistence API (JPA) with Hibernate as the provider. This abstracts raw SQL interactions, allowing developers to manipulate database records using Java objects (`Entities`). The database dialect is configured for PostgreSQL.

Database Relationship Strategy The database schema utilizes a mix of relationship strategies:

- **Many-to-One:** Used for hierarchical data like Users to Cities.
- **Many-to-Many (Implicit):** Used for Products and Categories via a hidden bridge table `productos_categorias`.
- **Many-to-Many (Explicit):** Used for Products and Stores via the `ProductsStoresEntity` class to handle stock attributes.

Query Pattern The system primarily uses **Spring Data JPA Query Methods** (method naming conventions) to automatically generate SQL. For complex aggregations, it utilizes **JPQL** (Java Persistence Query Language) via the `@Query` annotation.

2. Query Implementation Pattern Explanation

The standard flow for every query in this system follows the architectural pattern: **Entity Layer** → **Repository Layer** → **Service Layer** → **Controller Layer**.

2.1 Entity Layer

Entities are POJOs (Plain Old Java Objects) annotated with `@Entity`. They define the table structure.

- **Key Annotation:** `@Table(name = "x")` defines the physical DB table.
- **Relationships:** `@ManyToOne` annotations usually employ `FetchType.LAZY` to optimize performance by loading related data only when requested.
- **Serialization:** The `Serializable` interface is implemented to allow object transmission.

2.2 Repository Layer

Repositories extend `JpaRepository<Entity, ID>`.

- **Mechanism:** Interfaces are defined without implementation classes. Spring Data JPA creates the implementation at runtime.
- **Method Naming:** Methods like `findByLastName` are parsed by Spring to generate SQL (`SELECT ... WHERE last_name = ?`).
- **Custom Queries:** Complex logic uses `@Query("SELECT ...")`.

2.3 Service Layer

Services are annotated with `@Service` and `@RequiredArgsConstructor` for dependency injection.

- **Role:** Acts as a transaction boundary and pass-through wrapper.
- **Logic:** For explicit relationship entities (like `ProductsStores`), the service layer handles Java Stream mapping to transform bridge entities into domain entities.

2.4 Controller Layer

Controllers are annotated with `@RestController` and `@RequestMapping`.

- **Input:** Receives `@PathVariable` for IDs or `@RequestParam` for filter criteria.
- **Output:** Wraps results in `ResponseEntity.ok()` to ensure HTTP 200 responses.

3. Complex Queries Deep Dive

This section details the 14 advanced query implementations found in the project.

3.1 Query 1: Users by Last Name

Entity Analysis The `UserEntity` maps to the `usuarios` table. It contains a `lastName` field mapped to the column `apellido`.

Repository Implementation

- **Method:** `UserEntity findByLastName(String lastName);`
- **Strategy:** Exact match via Method Naming Convention.
- **SQL Generated:** `SELECT * FROM usuarios WHERE apellido = ?`

Service Layer The service simply delegates to the repository. Note that this returns a single `UserEntity`, implying business logic assumes last names are unique, or it returns the first match found.

Controller Endpoint

- **Mapping:** `@GetMapping("/last_name/").`
- **Parameter:** Uses `@RequestParam String lastName.`
- **Usage:** `GET /users/last_name/?lastName=Perez.`

3.2 Query 2: Users by City Name

Entity Relationship Mapping

- `UserEntity` has a `@ManyToOne` relationship with `CityEntity` named `city`.
- `CityEntity` has a `name` field.

Repository Join Strategy

- **Method:** `List<UserEntity> findAllByCity_Name(String cityName);.`
- **Strategy:** Property Traversal. The underscore `_` (or camelCase `CityName`) tells JPA to traverse from `User -> City -> Name`.
- **SQL Generated:** JPA generates an inner JOIN between `usuarios` and `ciudades`.

Service Flow The service returns a `List`, handling the potential for multiple users residing in the same city.

3.3 Query 3: Users by Department Name

Multi-Level Relationship This query traverses three tables: `User -> City -> Department`.

- `CityEntity` maps to `DepartmentEntity` via `department` field.

Repository Solution

- **Method:** `List<UserEntity> findAllByCity_Department_Name(String departmentName);`
- **Deep Traversal:** JPA intelligently builds a double JOIN:
 1. `usuarios` to `ciudades`
 2. `ciudades` to `departamentos`
- **Optimization:** This avoids the "N+1 select" problem by performing a single query with joins rather than fetching the user, then fetching their city, then fetching the department.

3.4 Query 4: Users by Name Fragment

Flexible Search Strategy

- **Method:** `List<UserEntity> findAllByFirstNameContaining(String nameFragment);`
- **Keyword:** `Containing` translates to the SQL `LIKE` operator with wildcards `%`.

Implementation details

- **SQL Generated:** `SELECT * FROM usuarios WHERE nombre LIKE '%fragment%'.`
- **Case Sensitivity:** By default, PostgreSQL `LIKE` is case-sensitive. To make this insensitive, the method would often be `findAllByFirstNameContainingIgnoreCase`.

3.5 Query 5: Products by Category

Bridge Table Queries (Implicit)

- **Relationship:** `ProductEntity` uses `@ManyToMany` with `CategoryEntity`.
- **Method:** `List<ProductEntity> findAllByCategories_Name(String categoryName);`
- **Strategy:** JPA automates the join across the `productos_categorias` bridge table defined in the `@JoinTable` annotation.

3.6 Query 6: Products by Price Range

Range Filtering

- **Entity:** `ProductEntity` contains a `Double price` field.
- **Method:** `List<ProductEntity> findAllByPriceBetween(Double minPrice, Double maxPrice);`
- **Keyword:** `Between` generates SQL: `price >= minPrice AND price <= maxPrice`.

Controller Endpoint

- **Mapping:** `@GetMapping("/price/")`.
- **Parameters:** Requires both `minPrice` and `maxPrice` as request parameters.

3.7 Query 7: Products by Creation Date

Temporal Query

- **Entity:** `ProductEntity` tracks `creationDate` using `@CreationTimestamp`.
- **Method:** `List<ProductEntity> findAllByCreationDateAfter(LocalDateTime date);`
- **Logic:** Filters records created strictly after the provided timestamp.

3.8 & 3.9 Queries 8 & 9: Products Ordered by Price

Sorting Strategies

- **Desc Method:** `findAllByOrderByPriceDesc()`.
- **Asc Method:** `findAllByOrderByPriceAsc()`.
- **Implementation:** These methods rely on the `OrderBy` keyword.
- **SQL:** Appends `ORDER BY price DESC` or `ASC` to the query. This is more efficient than sorting the list in Java memory.

3.10 Query 10: Products by Store Name

Bridge Table Queries (Explicit) This relationship is unique because it carries extra data (`stock`). It is mapped via `ProductsStoresEntity`.

- **Repository:** The query is run on `ProductsStoresRepository`:
`findAllByStoreEntity_Name(String storeName)`.
- **Service Logic (Mapping):** The Service layer performs the transformation from the bridge entity back to the product entity:

Java

```
productsStoresRepository.findAllByStoreEntity_Name(storeName)
    .stream()
    .map(ProductsStoresEntity::getProductEntity)
    .toList(); //
```

This separates the query logic (finding the association) from the presentation logic (extracting the product).

3.11 Query 11: Best Seller Products (Aggregation)

Aggregation & Grouping This requires a custom query because it involves calculating sums from a child table (`SalesProductsEntity`) and ordering by that sum.

- **Repository:** `SalesProductsRepository`.
- **Annotation:**

Java

```
@Query("SELECT sp.productEntity FROM SalesProductsEntity sp " +
        "GROUP BY sp.productEntity.id " +
        "ORDER BY SUM(sp.quantity) DESC " +
        "LIMIT 3")
List<ProductEntity> getTopThreeBestSellingProducts(); //
```

- **Analysis:**
 - `GROUP BY`: Aggregates sales rows by product.
 - `SUM(sp.quantity)`: Calculates total units sold per product.
 - `LIMIT 3`: restricts the output to the top performers.

3.12 Query 12: Sales by User ID

Foreign Key Filtering

- **Entity:** `SaleEntity` has a `@ManyToOne` relationship with `UserEntity`.
- **Method:** `findAllByUserEntity_id(Long userId)`.
- **Strategy:** Filters sales where the foreign key `id_usuario_fk` matches the parameter.

3.13 Query 13: Total Sales by Date

Scalar Aggregation

- **Repository:** `SaleRepository`.
- **Annotation:**

Java

```
@Query("SELECT SUM(s.total) FROM SaleEntity s WHERE CAST(s.sellDate AS
date) = :saleDate")
Double getTotalSalesByDate(@Param("saleDate") LocalDate saleDate);
```

- **Analysis:**
 - `SUM(s.total)`: Returns a single `Double` value, not a `List` of entities.
 - `CAST(... AS date)`: This is essential because `sellDate` is a `LocalDateTime` (timestamp), but the input is a `LocalDate`. It truncates the time portion for the comparison.

3.14 Query 14: Sales Over Amount

Threshold Filtering

- **Entity:** `SaleEntity` contains a `total` field.
- **Method:** `findAllByTotalGreaterThan(Double total)`.
- **Usage:** Used to find high-value transactions.

4. Bridge Table Queries Explanation

The system demonstrates two distinct ways of handling Many-to-Many relationships.

1. Implicit Bridge (Products <-> Categories)

- **Entity:** `ProductEntity`
- **Mapping:** `@ManyToMany`.
- **Structure:** Java does not have a class for `productos_categorias`. Hibernate manages the SQL inserts/deletes into this table automatically.
- **Querying:** Performed via traversal (`findAllByCategories_Name`).

2. Explicit Bridge (Products <-> Stores)

- **Entity:** `ProductsStoresEntity`.
- **Mapping:** Two `@ManyToOne` relationships (one to Product, one to Store).
- **Reasoning:** The bridge table contains an extra column: `stock` (`existencias`). This creates a "Relationship with Attributes" scenario, requiring a dedicated entity class.
- **Querying:** Queries must target the Bridge Entity (`ProductsStoresEntity`) first, then extract the Product or Store.